

Managemob über *ein einziges Panel* mit Coolify betreiben

Coolify ist eine selbst gehostete Plattform, die Deployment, Umgebungsvariablen, Domains und Monitoring in einem einzigen Dashboard vereint. Dieser Leitfaden erweitert den ursprünglichen Managemob-Leitfaden: Dein Code, dein KI-Assistent und Supabase bleiben genau dort, wo sie sind — nur die Veröffentlichungsschicht ändert sich. Und du schreibst weiterhin keine einzige Zeile Code von Hand.

EIN LEITFADEN DER PESSOA ACADEMY · SETZT DIE SCHRITTE 01-10 VON „GESTALTE MANAGEMOB NACH DEINEM BILD“ FORT

4

NEUE SCHRITTE
(11-14)

1

KONTROLLPANEL

0

VON HAND
GESCHRIEBENE
CODEZEILEN

~1h

DURCHSCHNITTliche
ZEIT

Was Coolify ist — und was es *ersetzt*

Coolify ist eine selbst gehostete Plattform für Deployment und Anwendungsverwaltung, die vereinfachen soll, wie Webanwendungen veröffentlicht, konfiguriert, aktualisiert und gewartet werden. Im Workflow des ursprünglichen Leitfadens wird Managemob lokal in Visual Studio Code entwickelt, über GitHub Desktop synchronisiert, für die Daten mit Supabase verbunden und auf Netlify veröffentlicht.

Die Einführung von Coolify ersetzt **weder** den Anwendungscode noch den KI-gestützten Arbeitsablauf noch Supabase als Datenschicht. Sie ersetzt die auf Netlify zentrierte Veröffentlichungsschicht durch eine einheitliche Betriebsumgebung: Deployment, Laufzeitkonfiguration, Domains, Umgebungsvariablen und Dienstüberwachung — alles verwaltet über ein einziges Kontrollpanel.

WARUM COOLIFY HINZUFÜGEN

Von einer Werkzeugkette zu *einem Ort*

Der ursprüngliche Leitfaden stützt sich auf eine Kette mehrerer Werkzeuge: GitHub speichert den Code, Visual Studio Code ist die Bearbeitungsumgebung, ein KI-Assistent schreibt den Code, GitHub Desktop synchronisiert die Änderungen, Supabase speichert Nutzer und Mobilitäten, und Netlify veröffentlicht die fertige Website online.

Dieses Modell ist wirksam für einen schnellen Start, verteilt die Verantwortung aber auf mehrere Oberflächen. Wer Managemob anpasst, muss sich zwischen lokaler Entwicklung, Repository-Synchronisierung, Verwaltung der Umgebungsvariablen, Build-Einstellungen, Deployment-Konfiguration und Domain-Verwaltung in getrennten Werkzeugen bewegen. Mit Coolify lebt alles, was **nach** dem Push passiert, in einem einzigen Dashboard, das dir gehört.

Umfang der neuen Architektur

EBENE 01 • UNVERÄNDERT

VS Code + KI-Assistent

Anpassung und Codeänderungen. Du beschreibst, die KI tippt — genau wie in den Schritten 01–10.

EBENE 02 • UNVERÄNDERT

GitHub-Repository

Versionskontrolle und Quellcodeverwaltung. GitHub Desktop synchronisiert weiterhin deine Commits.

EBENE 03 • UNVERÄNDERT

Supabase

Authentifizierung, Datenbank und Anwendungsdaten. Nichts wird verschoben, nichts wird migriert.

EBENE 04 • NEU

Coolify

Deployment, Laufzeitkonfiguration, Domains, Umgebungsvariablen und Betriebskontrolle — anstelle von Netlify.

Technische Voraussetzungen

Bevor du Coolify einführest, stelle sicher, dass die folgenden Bedingungen erfüllt sind. Sie entsprechen alle Schritten, die du im ursprünglichen Leitfaden bereits abgeschlossen hast.

- 01** Das **Managemob-Repository** (dein Fork) ist bereits auf GitHub verfügbar — Schritt 02.
 - 02** **Supabase** ist bereits konfiguriert: Schema importiert, Authentifizierung eingerichtet, Administrator angelegt — Schritt 03.
 - 03** Die Anwendung läuft korrekt in der lokalen Entwicklung: `npm install` abgeschlossen und `npm run dev` im Browser überprüft — Schritt 05.
 - 04** Die Datei `.env` bleibt über `.gitignore` von der Versionskontrolle ausgeschlossen — Schritt 04.
 - 05** Ein **Server oder eine virtuelle Maschine**, die Coolify hosten kann, steht zur Verfügung, da die Organisation ein selbst gehostetes Deployment-Modell wählt.
-

BONUS

Coolify ist Open Source und kostenlos selbst zu hosten. Die einzigen Kosten verursacht der Server, auf dem es läuft — eine kleine Cloud-VM reicht für den Anfang.

Richte deinen *Kontrollraum* ein

Warum

Coolify ist die Betriebsschicht, die die Veröffentlichung und Wartung von Managemob vereinfacht. Sie gibt der Organisation einen einzigen Ort, um das Repository zu verbinden, das Build-Verhalten festzulegen, Umgebungsvariablen zu speichern, Domains zuzuweisen und die laufende Anwendung zu überwachen.

Aktionen

- 01** Berechne einen **Server** oder eine **Cloud-VM** vor, auf der Coolify laufen wird (Ubuntu LTS ist die einfachste Wahl).
- 02** **Installiere Coolify** auf diesem Server gemäß dem von der Organisation gewählten Deployment-Modell, nach dem offiziellen Installationsskript auf coolify.io.
- 03** Öffne das Coolify-Dashboard und **erstelle ein Projekt oder einen Workspace** für Managemob.
- 04** Überprüfe, dass der Server **sowohl den Git-Provider als auch die Supabase-Endpunkte** erreichen kann, die die Anwendung verwendet.

PROMPT ZUM EINFÜGEN IN DEN KI-ASSISTENTEN

„Erkläre mir, was ich auf einem frisch aufgesetzten Ubuntu-Server prüfen sollte, bevor ich Coolify installiere: offene Ports, Firewall-Regeln, DNS-Einträge, die auf die Server-IP zeigen. Gib mir eine kurze Checkliste, ändere keine Datei.“

Gleicher Build, *neues Zuhause*

Warum

Coolify deploymt direkt aus deinem GitHub-Fork, genau wie es Netlify getan hat. Die Anwendung soll dieselben Build-Annahmen beibehalten, die bereits in Netlify verwendet wurden — im Code gibt es also nichts zu ändern, nur Einstellungen zu übernehmen.

Aktionen

- 01 Füge in deinem Coolify-Projekt eine **neue Anwendung** hinzu und verbinde sie mit deinem GitHub-Fork von Managemob (beim ersten Mal fragt Coolify nach einer Berechtigung, wie Netlify in Schritt 09).
 - 02 Stelle den Installationsschritt so ein, dass die Abhängigkeiten aus `package.json` verwendet werden (`npm install`).
 - 03 Setze den Build-Befehl auf `npm run build`.
 - 04 Setze das Veröffentlichungsverzeichnis auf `dist` (statische Website).
 - 05 Lass Coolify die Laufzeit-Bereitstellung über HTTP oder seinen integrierten Reverse Proxy übernehmen — SSL-Zertifikate werden automatisch verwaltet.
-

PROMPT ZUM EINFÜGEN IN DEN KI-ASSISTENTEN

„Überprüfe das SPA-Routing für ein statisches Deployment auf Coolify mit nginx oder dem Standard-Proxy: Welche Konfiguration brauche ich, damit das Aktualisieren einer Unterseite wie /dashboard keinen 404-Fehler zurückgibt?“

Bring die *Schlüssel* nach Coolify

Warum

Der ursprüngliche Leitfaden speichert die Supabase-Zugangsdaten während der Entwicklung in einer lokalen `.env`-Datei und verlangt, dieselben Variablen bei der Veröffentlichung erneut in Netlify zu definieren. Mit Coolify werden Produktionsvariablen in den **Anwendungseinstellungen der Deployment-Umgebung** verwaltet, nicht mehr in Netlify. Deine lokale `.env` funktioniert weiterhin für die Entwicklung — auf deinem Computer ändert sich nichts.

Aktionen

- 01 Öffne in Coolify deine Managemob-Anwendung → **Environment Variables**.
 - 02 Füge `VITE_SUPABASE_URL` mit der Project URL hinzu, die du in Schritt 03 aus Supabase kopiert hast.
 - 03 Füge `VITE_SUPABASE_ANON_KEY` mit dem anon public key derselben Seite hinzu.
 - 04 Markiere beide als **Build-Variablen**: Vite liest sie während des Builds, nicht zur Laufzeit.
 - 05 Starte ein **Deployment** und warte, bis der Build abgeschlossen ist. Öffne die URL, die Coolify dir gibt, und überprüfe, dass der Anmeldebildschirm erscheint.
-

WICHTIG

Committe niemals die `.env`-Datei auf GitHub und füge niemals den *Service-Role*-Schlüssel von Supabase in eine Front-End-Variable ein. In den Browser gehört nur der anon public key — die Row Level Security (Schritt 03) schützt deine Daten.

DOMAIN & GO-LIVE · 14

Deine Adresse, deine *Regeln*

Warum

Im Moment antwortet die Anwendung auf der von Coolify generierten URL. Um sie echten Nutzern zu übergeben, verbindest du deine eigene Domain und führst dieselbe Endprüfung wie in Schritt 10 durch — diesmal von der Coolify-URL aus.

Aktionen

- 01 Lege bei deinem DNS-Anbieter einen **A-Record** an (z. B. `mobility.pessoa-academy.com`), der auf die IP des Coolify-Servers zeigt.
 - 02 Öffne in Coolify die Anwendung → **Domains** → füge die Domain hinzu. Coolify stellt das SSL-Zertifikat automatisch aus.
 - 03 Aktiviere **automatische Deployments**: Jeder Push auf deinen GitHub-Fork löst einen neuen Build aus, genau wie bei Netlify.
 - 04 Wiederhole die **finale Checkliste** von der öffentlichen Domain aus.
-

Checkliste

- ☐ Die Anmeldung auf der öffentlichen Domain funktioniert.
 - ☐ Der Sprachumschalter funktioniert in allen 7 Sprachen.
 - ☐ Ich füge einen Teilnehmer hinzu, und nach F5 ist er noch da.
 - ☐ Ich generiere ein Word-Dokument mit einem Teilnehmer: Die Tags werden ausgefüllt.
 - ☐ Das Aktualisieren von /dashboard ergibt keinen 404-Fehler.
 - ☐ Das SSL-Zertifikat ist gültig (Schloss im Browser).
 - ☐ Ein Push aus GitHub Desktop löst ein neues Deployment in Coolify aus.
-

VOR DEM OFFIZIELLEN GO-LIVE

Aktiviere die E-Mail-Bestätigung in Supabase Auth wieder. Wechsle das DB-Passwort. Lade die echten Nutzer ein. Lösche den Test-Admin. Erstelle ein Backup — und mache es ab jetzt auch über das Coolify-Dashboard.

FAZIT

Eine Plattform, ein Panel, *weiterhin null Code* von Hand geschrieben

Visual Studio Code öffnet die Dateien. Der KI-Assistent schreibt den Code. GitHub Desktop synchronisiert. Supabase speichert die Daten. **Coolify bringt die Website jetzt online und hält sie am Laufen** — Deployments, Variablen, Domains und Monitoring an einem einzigen Ort, der dir gehört. Du koordinierst alles und triffst die Entscheidungen, die zählen. Das ist der Vibe Coding Workflow™, erweitert auf den Betrieb.

VIBE CODING • PESSOA ACADEMY • MANAGEMOB • COOLIFY • SELF-HOSTED • GITHUB • SUPABASE •
ERASMUS+ • LEARNING MOBILITY